

ABC458 F - Critical Misread 题解

题意概括

给定 K 个禁用串 s_i 。求长度为 N 的小写字母串中，有多少个串**不包含**任何 s_i 作为子串。答案对 998244353 取模。

其中：

- N 可达 10^9
- $K \leq 10$
- 每个禁用串长度 ≤ 10

解题思路

第一步：把“是否已经出现禁用串”做成自动机状态

这类“判断某些模式串是否出现过”的问题，标准做法是 Aho-Corasick 自动机。

我们把所有禁用串插入 Trie，再建失配指针。

对每个节点判断：

- 如果这个节点本身对应某个禁用串结尾，或者
- 它的失配链上已经到过禁用串结尾，

那么这个节点就是“坏状态”。

一旦转移到坏状态，就说明当前构造出的前缀已经出现过禁用串，不能再计入答案。

第二步：只在好状态之间转移

设自动机中的好状态个数为 S 。

定义一个 $S \times S$ 的转移矩阵 M ：

- $M[u][v]$ 表示从好状态 u 读入一个字母后转移到好状态 v 的方案数。

因为每次只能读一个字符，所以每个状态会向至多 26 个状态连边。

第三步：矩阵快速幂

设初始向量 dp 只有根节点这一位为 1。

读完 N 个字符后，状态向量就是：

$dp \times M^N$

把所有好状态的方案数加起来，就是长度为 N 且始终没有进入坏状态的字符串数量。

由于 N 很大，所以要用矩阵快速幂。

正确性说明

自动机的每个状态都表示“当前字符串后缀与哪些模式串前缀匹配”。

因此在逐个加入字符构造字符串时：

- 若转移到坏状态，说明某个禁用串已经作为子串出现；
- 若始终停留在好状态，说明到目前为止还没有出现任何禁用串。

于是：

- 每个合法字符串都会对应自动机上一条从根出发、长度为 N 、始终位于好状态的路径；
- 每条这样的路径也唯一对应一个合法字符串。

转移矩阵 M 精确记录了“一次加入一个字符”时好状态之间的转移关系，所以 M^N 恰好统计了长度为 N 的全部合法构造方式。

最终把所有终止好状态的方案数求和，就是答案。算法正确。

复杂度

设自动机节点数为 S 。由于 $K \leq 10$ 且每个串长度 ≤ 10 ，所以 S 很小，最多约 100。

- 建自动机： $O(S \times 26)$
- 矩阵快速幂： $O(S^3 \log N)$
- 空间复杂度： $O(S^2)$

参考实现

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const ll MOD = 998244353;
const int SIGMA = 26;
const int MAXS = 105;
```

```

using Matrix = vector<vector<ll>>>;

Matrix multiply(const Matrix &a, const Matrix &b){
    int n = (int)a.size();
    Matrix c(n, vector<ll>(n, 0));

    for(int i=0; i<n; i++){
        for(int k=0; k<n; k++){
            if(a[i][k] == 0){
                continue;
            }
            for(int j=0; j<n; j++){
                if(b[k][j] == 0){
                    continue;
                }
                c[i][j] = (c[i][j] + a[i][k] * b[k][j]) % MOD;
            }
        }
    }

    return c;
}

vector<ll> multiply_vec(const vector<ll> &vec, const Matrix &mat){
    int n = (int)vec.size();
    vector<ll> res(n, 0);

    for(int i=0; i<n; i++){
        if(vec[i] == 0){
            continue;
        }
        for(int j=0; j<n; j++){
            if(mat[i][j] == 0){
                continue;
            }
            res[j] = (res[j] + vec[i] * mat[i][j]) % MOD;
        }
    }

    return res;
}

```

```
int main(){
    long long n;
    int k;
    cin >> n >> k;

    static int nxt[MAXS][SIGMA];
    static int fail[MAXS];
    static bool bad[MAXS];
    memset(nxt, 0, sizeof(nxt));
    memset(fail, 0, sizeof(fail));
    memset(bad, 0, sizeof(bad));

    int tot = 0;

    for(int i=0; i<k; i++){
        string s;
        cin >> s;

        int p = 0;
        for(char ch : s){
            int c = ch - 'a';
            if(nxt[p][c] == 0){
                nxt[p][c] = ++tot;
            }
            p = nxt[p][c];
        }
        bad[p] = true;
    }

    queue<int> q;
    for(int c=0; c<SIGMA; c++){
        int v = nxt[0][c];
        if(v != 0){
            q.push(v);
        }
    }

    while(!q.empty()){
        int u = q.front();
```

```

q.pop();

bad[u] = bad[u] || bad[fail[u]];

for(int c=0; c<SIGMA; c++){
    int v = nxt[u][c];
    if(v != 0){
        fail[v] = nxt[fail[u]][c];
        q.push(v);
    } else {
        nxt[u][c] = nxt[fail[u]][c];
    }
}

}

vector<int> id(tot + 1, -1);
int cnt = 0;
for(int i=0; i<=tot; i++){
    if(!bad[i]){
        id[i] = cnt++;
    }
}

Matrix mat(cnt, vector<ll>(cnt, 0));
for(int i=0; i<=tot; i++){
    if(bad[i]){
        continue;
    }

    int from = id[i];
    for(int c=0; c<SIGMA; c++){
        int to_node = nxt[i][c];
        if(bad[to_node]){
            continue;
        }
        int to = id[to_node];
        mat[from][to] = (mat[from][to] + 1) % MOD;
    }
}

vector<ll> vec(cnt, 0);

```

```
vec[id[0]] = 1;

while(n > 0){
    if(n & 1LL){
        vec = multiply_vec(vec, mat);
    }
    mat = multiply(mat, mat);
    n >>= 1LL;
}

ll ans = 0;
for(ll x : vec){
    ans = (ans + x) % MOD;
}

cout << ans;

return 0;
}
```